gui_documentation

MIDAS GUI — User Documentation

Version: 1.0.0 **Application:** `midas-gui` (or `python -m midas_gui`) **Backends:** `midas_calibrate_v2`, `midas_integrate_v2`, `midas_calibrate`, `midas_hkls`, `midas_distortion` **Last updated:** 2026-07-29 (Data Viewer Ring simulation card reworked to support multiple materials at once — per-material checkbox row with a color swatch and a clickable name that opens a Material dialog for lattice/SG/preset editing and renaming; lattice fields now show 3 decimals and angle fields 2 decimals, down from 5/3)

> **Maintenance:** keep this document in sync with the code — whenever the workflow or a tab's controls change, update the relevant section here in the same change.

> **Tab status:** Tabs **0–4** (Data Viewer, Mask Builder, Calibrate, Calib. Refinement, Batch Integrate) are **verified** and ready to use. Tabs **5–8** (Corrections & Physics, PDF Analysis, Texture, Results & Export) are a **work in progress** and will be updated in the coming weeks.

Table of Contents

---

1. Overview and Architecture

The MIDAS GUI is a modular PyQt5 desktop application (up to 10 tabs) that exposes the scientific capability of the `midas_calibrate_v2` and `midas_integrate_v2` packages through a structured workflow. The intended order of use is:

```
Data Viewer (inspect) → Mask Builder → Calibrate → [Refine] → Batch Integrate
                                                                → Corrections preview
                                                                → PDF Analysis
                                                                → Texture / Pole Figure
                                                                → Pump Probe (TR-XRD)
                                                                → Results & Export
```

Modular tabs. Data Viewer, Mask Builder, Calibrate and Batch Integrate are always shown; the remaining tabs (Calibration Refinement, Corrections, PDF Analysis, Texture, Pump Probe, Results & Export) are optional and can be shown/hidden from **Settings ▸ Preferences ▸ Tabs**. By default only **Calib. Refinement** and **Pump Probe** are shown; Corrections, PDF Analysis, Texture and Results & Export ship **hidden** — turn them on when you need them. The choice is saved per-user (`ui.visible_tabs`) and applies immediately — see §15. Hidden tabs are only removed from the tab bar; they stay constructed, so cross-tab wiring and state are preserved.

Cross-tab shared state. When Tab 2 (Calibrate) produces a result it is automatically propagated to all downstream tabs. When Tab 1 (Mask Builder) computes a mask it is sent to the consuming tabs. Tab 3 (Refinement), if applied, re-broadcasts the refined geometry. No manual copying is needed.

All heavy computation runs off the GUI thread. Every operation that touches a MIDAS package (calibration, integration, mask training, PDF transform, gain training, field averaging, drift fitting) runs in a background `QThread` worker; the GUI stays responsive and log lines stream in real time.

Package layout.

```
midas_gui/
├── app.py           MainWindow, dark theme, crash diagnostics, main()
├── __main__.py      enables `python -m midas_gui`
├── style.py         Dioptas-inspired QSS theme + layout helpers
├── constants.py     calibrants, colormaps, dtype sentinels, default paths
├── helpers.py       image IO, geometry parsing, spec building, no-scroll widgets
├── widgets.py       ImageViewer / PickableImageViewer / ProfileViewer / FieldSelector / …
├── workers.py       all QThread workers + the integration core
├── calib.py         calibration-pipeline dispatch
├── dialogs.py       save dialogs
└── tab_*.py         one module per tab
```

Layout pattern. The four analysis tabs — **0 Data Viewer, 2 Calibrate, 3 Calib. Refinement, 4 Batch Integrate** — use a **three-panel layout**:

```
[ Data Loader | Parameters | Display / results ]
```

- **Data Loader (left).** A shared `DataLoaderPanel` for selecting the five inputs — **Data, Dark, Bright, Background, Mask** — each as a single file, a folder, or an HDF5 dataset (a container dropdown appears for HDF5). Frame controls live here too (Tab 0 navigator, Tab 2/3 frame index, Tab 4 frame range + stride).
- **Parameters (middle).** The tab's analysis controls.
- **Display / results (right).** Image viewer, plots, and/or a log panel.

The three panels are separated by **draggable splitter handles** — drag the `Data Loader | Parameters` and `Parameters | Display` boundaries to rebalance widths to taste. Each panel has a minimum width; drag past it and the panel shows a scrollbar rather than clipping its controls.

The remaining tabs (1, 5–8) keep the classic two-panel layout.

Corrections & mask (all four analysis tabs). Dark is averaged then subtracted; Bright is averaged then flat-field **divided** or **subtracted** (your choice); Background is averaged then subtracted; every enabled Mask source (files/folders plus the auto-added “Tab 1 mask”) is **unioned** into one composite mask that zeroes/ignores those pixels. Correction order: `(img − dark)` → bright → `− background` → clip ≥ 0.

---

2. Getting Started

`midas-gui` is **not on PyPI** — install it from source with conda.

```
git clone https://github.com/d-beniwal/MIDAS_GUI.git
cd MIDAS_GUI
conda env create -f environment.yml    # creates the 'midas-gui' env and installs the GUI
conda activate midas-gui
midas-gui                              # launch (equivalently: python -m midas_gui)
```

The GUI drives the MIDAS analysis backends (`midas-calibrate-v2`, `midas-integrate-v2`, `midas-calibrate`, `midas-hkls`, `midas-distortion`), which are also not on PyPI — point the `pip:` section of `environment.yml` at your MIDAS source before creating the environment (see the comments in that file).

Test data

Synthetic test data lives in the repo-root `test_data/` folder (git-ignored): - `calibrant_ceria.tif` / `.h5` — CeO_2 calibrant (Eiger2 500K, 1028×512, 75 µm pixels) - `nickel_tifs/` — 10-frame Ni scan, lattice expanding +0.1 %/frame - `nickel_stack.h5` — the same scan as one HDF5 stack (`exchange/data`)

Default geometry: λ = 0.39 Å, Lsd = 121 mm, pixel = 75 µm, BC = (10, 10) px. Every tab’s default file paths point at this data and **auto-load on startup when present**, so the app is usable immediately after checkout.

---

3. Tab 0 — Data Viewer

Purpose: inspect detector frames and do a quick radial integration — geometry-free (circle binning) by default, or a full tilt/distortion-aware integration when a calibration file is loaded. Produces no shared state for other tabs.

Data Loader panel (left)

Data, Dark, Bright, Background and Mask are all selected in the shared **Data Loader panel** (see §1): each accepts a file / folder / HDF5-dataset. The **Data** card here holds the **frame navigator** (slider, spin box, ◀/▶); **zoom/pan is preserved** as you step through frames (the view only auto-frames on a fresh load). Dark/bright/background and the composite mask are applied to the displayed image and to the radial integration. **Pan and zoom are bounded to the image** (roughly half an image-width of margin on each side) so scrolling/dragging cannot wander off into empty space or zoom out indefinitely; the bottom-left **A** (auto-range) button re-fits the image without leaving zoom “stuck” tracking the mouse.

Projection card

Collapse a stack to one image: **Max** (hot-pixel hunting), **Sum** (long-exposure equivalent), or **Average** (noise reduction), along a chosen axis (0 = across frames). **Skip frames** (default 1) ignores that many leading frames before projecting (e.g. 1 drops the first frame, 4 drops the first four) — useful when the opening frames are detector warm-up / shutter-transient exposures.

Calibration card (optional)

Load geometry from a **calibration `.json`, a MIDAS `paramstest.txt`, or a pyFAI `.poni`** (auto-detected). It fills BC, Lsd, pixel size and wavelength, unchecks "Beam centre = image centre", and refreshes the overlay and radial plot.

When a calibration file carries the **full geometry (tilts + distortion)**, the radial integration switches from simple concentric-circle binning to a **proper MIDAS-engine integration** that maps every pixel through the calibrated tilts and distortion (the same core as Batch Integrate) — so ring positions/intensities are geometry-correct, not just distance-from-beam-centre. The card's status line reports which mode is active. The binning geometry is built once and reused across frames (a fast `hard` kernel keeps the preview responsive); without a full-geometry file, the fast circle binning is used. If no calibration file is loaded but the Ring-simulation card's **ty/tz** tilt fields are non-zero, the radial integration still runs the tilt-aware MIDAS engine using a geometry built live from those fields (BC, Lsd, pixel size, wavelength) — so dialing in tilts here without a calibration file already produces a tilt-corrected profile, not just a tilt-shaped ring overlay.

Save JSON / Save params (.txt) / Save PONI (below the loader) export whatever geometry is currently in effect — the loaded calibration file if any, otherwise the one synthesized from the Ring-simulation widgets above — as a calibration file you can reload here, on the Calibrate tab, or in Batch Integrate. **Save params (.txt)** writes a standalone MIDAS `paramstest.txt`; **Save PONI** writes a pyFAI `.poni` (note: PONI's Rot1–3 convention cannot represent MIDAS's tx/ty/tz tilts, so tilts are **not** included in a `.poni` export — use JSON or `.txt` to keep them). Clicking any of the three before an image is loaded warns instead of writing a file (there is no detector size to write).

Intensity range card (radial integration)

Field	Description
Exclude out-of-range pixels	When on, pixels ≤ min or > max are drawn as a red overlay and excluded from the radial integration (removes gaps / hot / overflow).
pixel ≤ / pixel >	Lower / upper bounds. On load, the upper bound auto-fills to **max(99.99th percentile, 100000)**.

Ring simulation card

Overlays simulated Debye-Scherrer rings to check geometry. Supports **multiple materials at once** — e.g. a sample phase overlaid on a calibrant — each with its own lattice, visibility, and ring color. - **Material rows**: one row per material, each with a **checkbox** (show/hide that material's rings on both the image and the radial-integration plot), a **color swatch** button (click to open a color picker; the chosen color drives that material's ring lines and hkl labels on both plots), and the **material name** as a clickable (underlined) button. A ✕ button deletes the row — disabled when only one material remains, since at least one row is always kept. **+ Add material** appends a new row (default name `Material N`, generic cubic lattice, next color from a 10-color palette cycled by row order). A single default **Ni (FCC)** row is present at startup, matching the pre-multi-material behavior. - Clicking a material's name opens a **Material dialog**: an editable **Name** field, the **Preset** dropdown (CeO_2, LaB_6, Si, Al_2O_3, Cu, Ni, FCC-γFe, BCC-αFe, Au, Ag, Pt, W, Ti, or **Custom**), lattice **a, b, c** (3 decimals) on one row and **α, β, γ** (2 decimals) on the next, plus **SG #**, and a **Cubic (a=b=c, α=β=γ=90°)** checkbox that lets you enter only `a` for cubic crystals (b, c mirror a; angles fixed at 90°). Lattice fields are editable only for Custom. OK applies the name/lattice/preset back to that material's row (renaming here updates the row's displayed name); Cancel discards edits. - Geometry (λ, max 2θ, Lsd, pixel size, beam centre) is shared across all materials — only the lattice/space-group/color/visibility are per-material. Beam centre (auto = image centre, or manual BC_y/BC_z). The **λ** label is clickable (underlined) — click it for a menu of common K-edge foils (Pr, Sm, Yb, Lu, Hf, Ta, W, Re, Pt, Au, Pb, Bi) that sets λ to that element's K absorption-edge wavelength. The same clickable-λ menu is on the Calibrate and PDF tabs. The **px** label is likewise clickable — a menu of common detectors (GE 200 µm, Varex 150 µm, Pilatus 172 µm, Eiger 75 µm) sets the pixel size (also on the Calibrate tab, where it sets both pxY and pxZ). - Every field in this card steps by a fixed amount per up/down-arrow click (λ 0.01 Å, max 2θ 1°, Lsd 1 mm, pixel 0.1 µm, BC_y/BC_z 1 px, ty/tz 0.1°) — customizable from **Settings ▸ Preferences ▸ Data Viewer**. - **ty / tz** (detector tilt about the Y/Z axes, degrees) sit right below BC_y/BC_z — when non-zero, the simulated rings are forward-projected through the tilt geometry instead of drawn as plain circles, so the overlay shows the same non-circular ring shape a tilted detector actually produces. Leaving both at 0° reproduces the previous plain-circle rendering exactly. (Rotation about the beam axis, `tx`, leaves a full ring's shape unchanged, so it isn't exposed here.) - **Ring thickness** spin box (0.5–10 px) sets the line width of the simulated-ring overlay on the image; redraws immediately as you change it. - **Show rings / Labels** toggles; **Simulate rings** is a **live toggle** (not a one-shot click) — while on, the button turns **green** as a visual cue, and the overlay + hkl table recompute automatically whenever material, lattice constants, space group, or geometry (λ/Lsd/px/max 2θ) change. Beam-centre and ty/tz edits still just reposition the existing rings (their 2θ values don't depend on BC or tilt). Rings are drawn out to the full **max 2θ** you set, regardless of how far that places them from the beam centre (rings landing off-canvas simply aren't visible — they aren't silently dropped at some fixed pixel-radius cutoff). - **→ Send geometry to Calibrate** copies λ, pixel size, Lsd and beam centre into the Calibrate tab's detector + seed fields (the Calibrate tab has a matching ← **Data Viewer** button that pulls the same values).

Live Data card *(experimental)*

Sits **above the Data card**, collapsed by default behind its own title-bar checkbox — check it to reveal the live-PV controls, uncheck to hide them again (unchecking also stops an active stream, so a hidden card can never be left silently connected). Subscribes directly to an EPICS PVA detector-image PV (NTNDArray) and renders each frame **inline in the Data Viewer's own image pane** — the same view used for files/folders/HDF5 stacks. Because it feeds the normal frame pipeline, all existing Data Viewer analysis keeps working live: dark/bright/background/mask corrections, the intensity-range mask, beam-centre picking, and the radial integration plot all update as new frames arrive. - **Dependency:** `pvapy` (the EPICS pvAccess client library) is a required, pinned dependency (`pyproject.toml` / `environment.yml`), installed automatically with the rest of the GUI's stack — no separate extra to install. If somehow missing from the active environment, **Start** shows an install hint instead of failing outright. - **Live PV** field is an editable dropdown: pick a known device by name (the list comes from **Preferences ▸ Devices**, see below) to fill in its full PV automatically, or type any other PV by hand (placeholder shows an example, `20IDFF:Pva1:Image`). **Start** / **Stop** buttons and a status line (stopped / waiting for PV / connected / streaming with frame id / error). GUI updates are throttled to the tab's existing ~16 fps debounce, so a fast PV update rate doesn't overwhelm the interface. - **Stopping live streaming does not auto-reload the previous file/folder/HDF5 source** — the Data card below gets a small ⟳ **Reload** button next to its path field for exactly this: click it after Stop to restore the static data that was loaded before streaming started. - **Ring overlay is static per-frame while streaming**: rings drawn by "Simulate rings" (Ring simulation card) keep rendering on top of each new live frame at their already-computed geometry — arrival of a new frame does **not** by itself trigger a recompute (ring radii don't depend on frame data). With Simulate rings' live toggle on, changing a material/lattice/geometry field still recomputes and redraws immediately. **Stop** leaves the last received frame on screen. Closing midas-gui also stops any running stream. - **Colormap/level changes**

persist across frames: dragging the histogram's level range (or the cmap dropdown) is remembered and reapplied to every new incoming frame instead of being reset to the vmin%/vmax% percentile defaults; editing vmin%/vmax% explicitly switches back to auto-levels.

Beam-centre picking (on the image)

The image viewer has **Pick BC** (single click sets the beam centre) and **Pick Ring** (click ≥3 points on a ring; a circle fit estimates the beam centre). Either updates BC_y/BC_z and re-runs the overlay + radial integration. Pick Ring's picked points and fitted circle are drawn in **blue**, distinct from the **amber** Simulate-rings overlay so the two aren't confused when both are visible on the same image.

Top-N brightest pixels (image toolbar)

A **Top-N pixels** toggle button (with an **N** spin box) sits on the image toolbar. When on, the **N highest-intensity pixels** of the current frame are marked with a crosshair inside a translucent circle (cyan) centred on each pixel, and the Intensity statistics panel switches to show the statistics of just those N pixels ("Top N pixels"). It follows frame changes while active. Clicking the button again removes the markers and restores the normal statistics. Useful for quickly locating saturation / hot pixels.

An **I >** checkbox next to the N spin box enables an optional intensity floor (a field to its right, editable once the checkbox is on): when set, pixels at or below that value are excluded from ranking entirely — never marked, never counted toward N — so a low-signal frame can't be forced to mark N pixels that aren't actually meaningful. If fewer than N pixels clear the threshold, fewer than N markers are drawn (down to none).

Intensity statistics (left panel, bottom)

At the bottom of the Data-Loader panel, in a **draggable pane** below the loader cards — grab the splitter handle above the panel to make the statistics area taller or shorter. It holds a **histogram** of the intensity distribution (full range, log-y toggle) with a **textbox** beneath it reporting N (pixel count) and the **p70 / p90 / p99 / p99.9 / p99.99** percentiles — each with the **number of pixels above** that value. The histogram's lower-left corner is fixed at x = 0, y = −2 and both axes rescale to `(0, xmax)` / `(−2, ymax)` on every refresh (frame change, scope change, projection, new data). It reflects the **corrected** image (dark / bright / background) with masked pixels (file masks + the intensity-range mask) excluded, and updates live as any of those change. A scope selector switches between the **current frame** (per the slider) and **All frames** (combined over the whole stack/folder). When a **Projection** is active the panel shows the projected image's statistics (the scope selector is disabled); when **Top-N pixels** is active it shows those pixels' statistics.

A small **"A"/"M"** button pair sits in the histogram's bottom-left corner (replacing pyqtgraph's native auto-range corner button), same control as on the radial-integration plot below — see **Manual axis limits (A/M toggle)** under that section for the full behavior (native right-click "Manual" min/max fields, persistence across live updates, reclick-to-reset). In **Manual**, the histogram holds those limits instead of auto-rescaling to `(0, xmax)` / `(−2, ymax)` on every refresh (new frame, scope change, Top-N toggle).

Radial integration plot (bottom-right)

Below the image is a live **azimuthal average about the beam centre** (`R bin`, `Integrate`, and an `Auto` toggle that recomputes on frame/BC/mask change) — geometry-free circle binning by default, or the tilt/distortion-aware MIDAS engine when a full geometry is in effect (a loaded calibration file, or non-zero ty/tz in the Ring-simulation card — see the Calibration card above). Peak/ring markers overlaid on the plot use the ring's true 2θ, so they line up with the profile in either mode. **Clicking a radius on the plot draws the matching ring (magenta) on the image.** Axis units switch between R (px) / 2θ / Q; the **X-axis lower bound defaults to 0**.

The default view always auto-fits the current profile's X/Y extent — it never gets stuck showing a stale or unrelated range from an earlier profile. If you manually zoom or pan (drag, wheel-zoom, box-zoom, or the axis context menu), that exact view is **preserved across parameter changes** (new frame, changed BC/tilt/Lsd, etc.) instead of snapping back to full range every time the profile updates — the curve redraws in place under your current zoom. A manual zoom is only cleared when it would no longer make sense: switching the X-axis unit (R/2θ/Q) drops the remembered X range, and toggling **Log Y** drops the remembered Y range (both because the old numbers belong to a different scale). **Pan and zoom are always bounded to the current profile's extent** (a margin around its X/Y range) so you cannot drag or scroll off into empty space. The splitter handle above this panel (between it and the image view) is wider than a default Qt splitter, to make it easier to grab.

Manual axis limits (A/M toggle)

A small **"A"/"M"** button pair sits in the plot's bottom-left corner, in the spot pyqtgraph's native auto-range button normally occupies (that native button is hidden in favor of this pair). **A** (default) is the auto-fit behavior described above. **M** switches to **Manual**, which holds exactly the limits set via each axis's own **native right-click menu** — right-click the plot, open **X axis** or **Y axis**, pick **Manual**, and type a min/max (this is stock pyqtgraph, not a MIDAS-specific control). Once **M** is active, the plot's axes show **exactly** those typed values (e.g. entering `0` shows `0`, not a padded/rounded value) and hold them through every live-acquisition redraw — Manual mode stops the plot from re-fitting or re-clamping its range on each new frame, and editing the min/max fields again takes effect immediately, live acquisition or not. Switching to **A** does not discard the typed values — clicking **M** again restores exactly what was last entered. **Reclicking the already-active button resets the view to that mode's default**: **A** forces an immediate re-fit to the current profile (discarding any manual pan/zoom drift), and **M** snaps the view back to the held manual limits (discarding any drift from panning around while still in Manual).

---

4. Tab 1 — Mask Builder

Purpose: identify and exclude bad pixels. The final mask is a uint8 array (1 = bad) broadcast to Tabs 2, 3, 4, 5. Browsing an image or a mask loads it immediately. Pointing the **Image** field at an `.h5` file reveals a **Dataset** dropdown (auto-populated with the file's datasets and shapes, preferring a ≥2-D dataset) — the same pattern used by the Stack field below it and by the Data Loader panels elsewhere in the app.

Section 1 · Threshold mask (always applied)

`pixel ≤ lower | pixel > upper`. The upper bound auto-fills from the data type on load (e.g. 1,048,575 for uint20 Eiger, 4,294,967,295 for uint32).

Section 2 · Statistical auto-mask

Two **independently selectable** methods — enable either or both: - **Spatial outlier** — robust local-anomaly detection (5×5 median residual → 15×15 local MAD → Z-score → hot/dead/saturated gates). Uses a **temporal median** over the stack folder when provided (cleaner reference), else the single frame. Controls: `K_σ` (6.0), `Hot` (1.5), `Dead` (0.5). - **Temporal constancy** — flags frozen pixels whose frame-to-frame std is below `Frozen × Q75(std)` (0.05); needs a stack of ≥2 frames.

A shared **stack source + stride** feeds both (the temporal median for spatial, the frame stack for temporal constancy). The stack can be a **folder / `*.tif` glob**, or a **single HDF5 file whose 3-D dataset is a time sequence of images** — for an `.h5` a **Dataset** selector appears (auto-populated, 3-D datasets preferred). *All σ / K_σ / n_σ fields in this tab accept any value — there is no upper limit.*

Section 3 · Spatial spike rejection

Laplacian high-pass single-pixel-spike detector; `n_σ` threshold (5.0).

Section 3b · Cosmic-ray rejection (temporal)

Per-pixel temporal σ-clip across a ≥3-frame stack; anomalies OR'd across frames. `n_σ` (5.0).

Section 4 · Calibration-based masks (need a Tab 2 result)

- **Azimuthal σ-clip** — flags (R, η) cells deviating from the azimuthal mean.
- **Learnable mask** — differentiable per-pixel mask trained against an η-uniformity loss (`steps`, `lr`, `sparsity`).

Draw mask tools

Interactive rectangle / oval / circle / polygon / annulus / point tools, live-draggable; **Apply shapes → mask** rasterises them (OR'd with the computed mask). **Clear shapes** removes them.

Save / Load

Save the combined mask as TIFF (0 = good, 1 = bad); loading a TIFF applies immediately.

---

5. Tab 2 — Calibrate

Purpose: determine detector geometry (Lsd, BC, tilts, distortion, wavelength) from a calibrant pattern.

Data Loader panel (left)

The calibrant frame (Data + Frame index), Dark, Bright, Background and Mask are selected in the shared **Data Loader panel** (see §1). Each Dark/Bright/Background field is averaged over a chosen index range; Bright offers **Flat-field divide** or **Subtract**; all mask sources (plus the auto-added Tab 1 mask) are unioned. Dark is passed to the calibration pipeline; bright/background are applied to the calibrant before calibration and post-calibration integration.

Detector, seed & Load calibration file

The **Detector & Calibrant** card sets λ, pixel size(s) and detector transforms; the **Initial seed** card sets the LM starting point (BC_y, BC_z, Lsd, **and tilts tx/ty/tz**). **Load calibration file…** (top of the Detector card) reads a MIDAS **paramstest `.txt`**, a calibration **`.json`**, or a pyFAI **`.poni`** (auto-detected) and fills λ, pixel size, and the seed BC + Lsd — a fast way to start from a previous calibration or a known geometry. (The synthetic test data ships `test_data/calibration_synthetic.{json,txt,poni}` as a ready example.) ← **Data Viewer** (next to *Load calibration file…*) pulls λ, pixel size, Lsd, beam centre and the Data Viewer's ty/tz tilt fields straight from the Data Viewer tab into the same fields (BC and Lsd land in the seed card; ty/tz land in the seed tilt fields).

A **Feed result back to seed** checkbox (on by default) copies the optimized BC / Lsd / tilts / distortion of each run back into the seed fields, so a follow-up run starts from the previous solution. *Seed tilts are honoured by the Four-stage / advanced pipelines; the One-shot / First-time paths seed tilts only if the installed backend exposes initial-tilt options.*

Average frames

For a multi-frame source (HDF5 / folder), **Average frames into a single image** builds the mean of a frame range and calibrates on that. **start** / **end (0 = all)** select the range and **skip** uses every Nth frame while averaging (e.g. skip = 2 averages every other frame). The card is disabled for single-frame sources; the preview updates live as the options change.

Pipeline selector

One-shot (default) · First-time · Four-stage · Bayesian (Laplace σ) · Joint-cake. *For trustworthy tilt/strain, prefer Four-stage or First-time — One-shot/Bayesian can report a spurious self-compensated tilt on weakly-tilted data.*

Refine flags

Which parameters vary: Lsd, BC, ty, tz, tx, Wavelength, plus a “Residual map” build toggle. The **Distortion (n/15)** checkbox has a companion **…** button that opens a per-coefficient dialog: the 15 distortion coefficients are grouped by η-fold (isotropic radial + folds 1–6), and named **preset modes** (None · Isotropic only · Iso + up to 2-fold · Iso + up to 4-fold · All (15)) auto-select whole ladders. The checkbox label shows how many coefficients are selected. *Per-coefficient control applies to the Four-stage / advanced pipelines; the One-shot path refines all-or-none.* Advanced (E-M / LM iters, device, output dir) and Multi-panel detector groups are collapsible.

Live threshold slider

Zeroes calibration-image pixels below the slider value (background suppression for ring-finding); the preview updates instantly.

Pick BC / Pick Ring

Click the image to seed the beam centre (single click) or fit a ring (≥3 clicks); the Pick Ring points/fit are drawn in blue, distinct from the amber Simulate-rings overlay.

Predicted-ring overlay (image toolbar)

After a run, the calibrant’s predicted ring radii are drawn in **lime** with a red/yellow beam-centre marker. **Show rings** toggles the overlay; **Corrected** redraws the same lime rings reshaped through the fitted tilt (tx/ty/tz) instead of as plain circles, so you can see how much the geometry actually deviates from an untilted detector — the rings still look like the same thin curves, just bent, rather than a separate scattered point-cloud.

Run / Abort

Run Calibration launches the worker; **Abort** terminates it and frees the slot so you can immediately start a new run (the calibration is one uninterruptible library call, so abort hard-terminates the worker thread rather than waiting).

Results (right panel, bottom tabs)

Radial Profile (with ring markers and an **Azim. avg** selector — see Tab 4), Ring Residuals bar chart, **Results**, and Log. Integration runs automatically after calibration. The **Results** tab shows the *complete* parameter set exactly as it is written to `paramstest.txt` (Lsd, BC, tx/ty/tz, the distortion coefficients, Parallax, Wavelength, px, NrPixelsY/Z, RhoD, SpaceGroup, LatticeConstant) as **plain text laid out in multiple columns** so the wide-but-short panel stays readable — no table widget. The distortion `p0–p14` slots are labelled with their coefficient names (iso_R2, a1, phi1, …). A strain/timing line sits below. → **Send to Data Viewer** pushes the *full* calibrated geometry (λ, pixel size, Lsd, beam centre, **tilts and distortion**) into the Data Viewer tab — where it drives the tilt/distortion-aware radial integration, not just circle binning — the reverse of the Data Viewer's "→ Send geometry to Calibrate". The bottom tab area is fully resizable (drag the horizontal splitter) and the Log fills its tab.

Export

Save calibration.json and **Save paramstest.txt** (standalone or from a template).

---

6. Tab 3 — Calibration Refinement

Purpose: optional post-calibration geometry refinement against an **η-uniformity** criterion (rings should be azimuthally uniform). Uses a **derivative-free Nelder-Mead** optimiser (the differentiable integrator returns NaN geometry gradients at the beam-centre singularity on this build).

The sample frame and Dark/Bright/Background/Mask are selected in the shared **Data Loader panel** (see §1) — the refinement runs on the corrected image. Middle-panel controls: calibration source (Tab 2 result, or start-from/continue radios), parameters to refine (BC_y, BC_z, Lsd, ty, tz), optimiser + iterations. **Run Refinement** shows a live loss curve; **Apply** broadcasts the refined geometry downstream. If Apply is never clicked, the Tab 2 result flows through unchanged.

---

7. Tab 4 — Batch Integrate

Purpose: integrate a stack of frames into 1-D profiles using the calibrated geometry and optional corrections.

Data Loader panel (left)

The streaming **Data** source (folder/glob or HDF5 dataset) with **frame range + stride**, plus Dark/Bright/Background and Mask, are selected in the shared **Data Loader panel** (see §1). Dark/bright/background are applied per frame; all mask sources are unioned.

Calibration source (middle)

- **From Tab 2** — the calibration (or refined) result.
- **From file** — a **calibration `.json`, MIDAS `paramstest.txt`, or pyFAI `.poni`** (auto-detected; both GUI and MIDAS-pipeline json key styles supported). Entering a path auto-selects this option.

Calibration values (middle, read-only)

A **Calibration values** card shows the geometry actually in use — λ, Lsd (mm), BC_y/BC_z (px), tilts tx/ty/tz (°), pixel sizes, detector size, and a distortion summary (number of non-zero coefficients). It repopulates whenever a calibration arrives from Tab 2 (or Tab 3 refinement, or the Data Viewer via Tab 2) **and** when a calibration file path is entered, parsing the file directly so you can confirm the numbers before running. A note line reports the active source (or any file-read error).

Integration

Field	Description
Kernel	Hard (fastest) · Subpixel K=2 · Subpixel K=4 · Polygon (exact).
R bin (px) / η bin (°)	Radial and azimuthal bin sizes.
Azim. avg	How the (η, R) cake becomes a 1-D profile: **Pixel-weighted** (default) `Σ(mean·count)/Σ(count)` — independent of η-bin size and robust to partial azimuthal coverage / **off-detector beam centres**; or **η-bin mean (legacy) —** the unweighted mean of per-η-bin means, which can distort with a coarse η bin when the beam centre is off the detector.
Per-bin variance (σ)	Error model poisson / azimuthal / hybrid (ignored when corrections are on → σ = √I).
Q-uniform bins	Integrate in R then rebin onto a uniform-Q grid (Qmin, Qmax, ΔQ).

Physics corrections

Polarization and solid-angle (pixel-domain, via `integrate_with_corrections`).

Monitor normalisation

Divide each processed frame's profile/σ by a per-frame scalar from a text file (one value per *processed* frame).

Drift correction (long scans)

Per-frame geometry interpolated from an anchor JSON (`{frame_idx: {Lsd, BC_y, BC_z}}`) with spline/linear/constant parametrization; **Fit trajectory** builds it. Each frame is then integrated with its own geometry.

Run / Abort / Clear

Start Integration / **Abort.** Abort first asks the worker to stop cleanly between frames (keeping frames already written); if it does not stop promptly it force-terminates and frees the slot. Completed frames' output files are preserved. **Clear results** removes the profiles/plots computed **this session** for the current data (waterfall, stacked profiles, and the integrated-frame tracking) so a fresh integration can start — it stops any active monitor but does **not** delete raw data or any files already written to disk.

Live folder monitoring (MONITOR)

The **MONITOR** button at the bottom of the left Data Loader panel (folder/glob sources only) starts a live watch: while active it turns **green** and the GUI polls the data folder for **new** TIFF frames, integrating each one **as it appears** and adding it to the display. It does **not** re-run the whole batch — it reuses the already-built **detector map** (the binning geometry / pixel-count cakes; reused from a prior *Start Integration* run when the calibration, kernel, bins, mask and folder are unchanged, or built once on first use) and integrates **only the new files** (tracked by frame id, so frames already shown are skipped). New frames honour the current kernel, corrections, Dark/Bright/Background, mask and Q-uniform settings, and are saved to the output folder when a 1-D format is selected. Click MONITOR again to stop; starting a fresh *Start Integration* also stops it. (Distinct from **Monitor normalisation** above, which divides profiles by a scalar file.)

Output formats

CSV (R,I,σ) · XYE (2θ) · FXYE (centideg) · DAT (Q) · HDF5 (full stack) · 2D-CSV (η×R cake). Right panel: live **Waterfall** and **Stacked profiles** — both have an **x** selector to show the axis in **R (px) / 2θ (°) / Q (Å^{-1})** (converted from the run's calibration).

The **Stacked profiles** view is a publication-quality plot: each curve tags its **source file name inline, just below the curve at its left edge** (toggle with the *Labels* checkbox; an optional corner *Legend* is also available). A **theme** selector switches between two saved presets: - **White (publication)** *(default)* — white background, **point + line** markers, a colour-blind-friendly categorical palette (matplotlib *tab10*), a boxed frame and a light grid — ready to drop into a paper. - **Dark** — the classic on-screen look (dark background, line-only, vivid golden-angle colours).

An **x** selector plots the axis in **R (px)**, **2θ (°)** or **Q (Å^{-1})** (converted from the run's calibration). The **Grid** checkbox (off by default) toggles the horizontal + vertical grid. The *spacing* box shifts successive curves vertically (0 = overlay). Top-right controls adjust the **line width** (`− line +`), **symbol size** (`− sym +`, for the point+line markers — also turns markers on if a line-only theme is active) and **label font size** (`− font +`).

---

8. Tab 5 — Corrections & Physics *(work in progress)*

Preview physics corrections on a single frame (auto-loads on browse). Pixel-domain: polarization, solid angle, empty subtraction. Profile-domain: cylindrical absorption (μR), Compton subtraction (composition:fraction). **Compute corrected profile** overlays corrected vs uncorrected and shows the correction factor vs 2θ.

Also hosts **per-pixel gain training (LearnableGain)**: from a clean reference frame and a drifted frame, learn a spatial gain map `g_i = 1 + scale·r_i` by minimising `MSE(profile) + unity·Σ(g−1)² + smooth·TV(g)`; save as NPZ and apply with `corrected = raw / gain_map`.

---

9. Tab 6 — PDF Analysis *(work in progress)*

Polyatomic **total-scattering** reduction: I(Q) → Faber-Ziman structure function S(Q) → pair-distribution G(r), powered by the `midas_pdf` backend (real composition-weighted normalization, Compton subtraction, end-to-end σ propagation, and optional differentiable scale/background refinement). This replaces the earlier monoatomic, composition-free `F(Q)=Q·(I/bg−1)` approximation.

Background — what I(Q), S(Q) and G(r) mean

PDF analysis is three transforms that move from *what the detector measured* to *where the atoms are*. They are exactly the three stacked plots (top → bottom).

- **I(Q) — measured scattered intensity.** Intensity vs. momentum transfer `Q = 4π·sin(θ)/λ` ($Å^{-1}$), a wavelength-independent rescaling of the angle 2θ; high Q = wide angle = fine spatial detail. Obtained by azimuthally integrating the detector rings to a 1-D curve. I(Q) is **not** pure structure — it mixes the interference (coherent) signal with big smooth backgrounds: per-atom self-scattering (form factors `⟨f²⟩`) and inelastic **Compton** scattering.

- **S(Q) — structure function.** I(Q) with the self-scattering and Compton removed and normalized away (the **Faber-Ziman** step), leaving only interference: `S(Q) = [I_coh − (⟨f²⟩ − ⟨f⟩²)] / ⟨f⟩²`, where `⟨f⟩, ⟨f²⟩` are composition-weighted atomic form factors (hence the **Composition** input). S(Q) oscillates about **1** and → 1 at high Q where correlations wash out (the dashed guide line). `F(Q) = Q·[S(Q)−1]` is the same thing re-weighted by Q to emphasize the structurally rich high-Q oscillations.

- **G(r) — reduced pair distribution function.** The real-space answer: a histogram of interatomic distances, via a sine Fourier transform `G(r) = (2/π)·∫ Q·[S(Q)−1]·sin(Qr) dQ`. A **peak at r** means many atom pairs are separated by that distance; the **first peak is the nearest-neighbour bond** (Ni ≈ 2.49 Å, CeO_2 Ce–O ≈ 2.34 Å), later peaks are further coordination shells. Below the first bond `G(r) = −4πρ₀r` (nothing can be closer) — a constraint that needs the number density ρ_0. Because it uses *total* scattering (Bragg **and** diffuse), the PDF captures local structure even in disordered / nanoscale / amorphous materials, unlike a Bragg-only Rietveld fit.

```
detector image / I(Q) file
        │  azimuthal integration
        ▼
    I(Q)   raw intensity = interference + form factors + Compton      (top plot)
        │  subtract Compton, subtract Laue (⟨f²⟩−⟨f⟩²), divide by ⟨f⟩²  (needs composition)
        ▼
    S(Q)   pure interference, oscillates about 1                      (middle plot)
        │  Fourier sine transform over [Qmin, Qmax] with a window (Lorch)
        ▼
    G(r)   interatomic distances; peaks = coordination shells         (bottom plot)
```

The **±1σ band** on G(r) is the counting uncertainty σ propagated from I(Q) through every step, so real peaks can be told from noise. The **G/g/T/R** family (Output selector) is the same information re-weighted: **G** oscillates about 0 (refinement standard); **g** → 1 at large r; **T** is the total correlation; **R** is the radial distribution whose *peak area = coordination number*. g/T/R all need ρ_0.

I(Q) source (choose one): - **Integrate detector frame** — a calibrated frame is integrated (hard/polygon binning, Poisson σ) and mapped to Q, exactly as before. Uses the Tab 2 calibration (λ, geometry) and any Tab 1 mask. - **Load I(Q) file** — a pre-integrated 2- or 3-column `Q, I, σ` text/CSV (comment/header lines tolerated). The tab **opens on this source by default**, pointed at real data in `test_data/pdf_real/` (`iq_Nickel.csv`; also CeO_2, IPA, Kapton at λ 0.1839) — just press **Compute G(r)** for the Ni PDF (first peak ~2.49 Å). Synthetic known-answer examples are in `test_data/pdf_synth/` (`nickel_iq.csv`, `ceo2_iq.csv`). See each folder's `README.md` for per-sample composition / ρ_0 / Q-range settings.

Calibration. The geometry/wavelength comes from Tab 2 automatically, or use **Load calibration file…** to read a MIDAS `paramstest.txt`, a `.json`, or a pyFAI `.poni` (auto-detected). This is required for *Integrate detector frame* and also sets λ used by *Load I(Q) file* mode.

Sample. Composition (e.g. `Ni` or `C:3,H:8,O:1`; ions like `Ni2+` allowed), number density ρ_0 (atoms/Å³; needed for refinement and for g/T/R), wavelength λ (auto-filled from calibration), and a **Compton subtraction** toggle.

Normalization. Optional **Refine scale + background** (L-BFGS) — reveals a background polynomial degree (0–3), a low-r cutoff `r_min` (Å), and an iteration count. Refinement requires $\rho_0 > 0$. It fits scale and a smooth b(Q) against two model-free constraints: high-Q ⟨S⟩→1 and $G(r)=-4\pi\rho_0 r$ below `r_min`.

Output. Bottom-plot convention family — **G(r)** (reduced PDF), **g(r)** (pair distribution), **T(r)** (total correlation), or **R(r)** (radial distribution, whose peak integral is the coordination number); g/T/R need ρ_0. Middle plot toggles between **S(Q)** (with the S=1 guide) and the true reduced structure function **F(Q)=Q(S−1)**.

Right panel: I(Q) (+ refined background overlay), S(Q)/F(Q), and the selected G/g/T/R with a shaded **±1σ** band. **Save G(r)** writes a three-column `r, G(r), σ` file (diffpy-CMI / PDFgui / RMCProfile compatible); **Save S(Q)** writes `Q, S(Q)`.

Functions behind the tab

The tab is a thin UI over a background worker and the `midas_pdf` backend.

UI — `PDFTab` (`midas_gui/tab_pdf.py`)

Method	Role
`_build_ui`	Builds the I(Q)-source / Sample / Normalization / Output groups and the three stacked plots.
`set_calibration(result, source)`	Receives a Tab-2 result (or a loaded geometry) → sets λ and enables Compute.
`_load_calib_file`	Loads a `paramstest` / `.json` / `.poni` → builds a calibration result → `set_calibration`.
`_load_img`	Loads a detector frame for *Integrate detector frame* mode.
`_run`	Validates inputs, assembles the `cfg` dict, and starts a `PDFWorker`.
`_on_done` / `_redraw_mid` / `_redraw_bottom`	Draw I(Q)+bg, the S(Q)↔F(Q) toggle, and the G/g/T/R curve with its ±1σ band.
`_save_gr_file` / `_save_sq_file`	Export `r,G,σ` and `Q,S`.

Worker — `PDFWorker` (`midas_gui/workers.py`) runs off the GUI thread:

Function	Role
`_acquire_iq`	Produces `(q, I, σ)` — either by integrating the frame (image mode) or by reading a file (`_load_iq_file`, tolerant of comma/space and 2- or 3-columns).

Function	Role
`run`	Trims to `[Qmin,Qmax]`, builds `Composition`, calls the reduction (plain or refine), computes `F(Q)` and the G/g/T/R family, emits the result dict.

Image-mode integration reuses the shared core `_build_spec`, `build_geom`, `integrate_frame`, `axis_conversions` (same path as the other tabs); geometry-file parsing uses `geometry_fields_from_file` / `result_ns_from_geometry_file` (`midas_gui/helpers.py`).

Reduction — `midas_pdf` via `midas_gui/pdf_backend.py` (re-exported symbols):

Function	Does	Maps to plot
`Composition(fractions, number_density)`	Composition layer: `⟨f⟩, ⟨f²⟩` form-factor averages, Laue term `⟨f²⟩−⟨f⟩²`, and per-atom Compton.	—
`faber_ziman_S(I, q, comp, …)`	I(Q) → S(Q) with σ (subtract Compton/Laue, divide by `⟨f⟩²`).	S(Q)
`structure_function_F(q, S)`	`F(Q) = Q·(S−1)`.	F(Q)
`fourier_sine_transform(q, S, r, window)`	S(Q) → G(r) with σ (Lorch / none window).	G(r)
`i_of_q_to_Gr(q, I, comp, r, …)`	End-to-end I(Q) → G(r) (composes the two above); the default Compute path.	S(Q) + G(r)
`refine_normalization(q, I, comp, r, …)`	L-BFGS fit of scale + background polynomial against high-Q ⟨S⟩→1 and low-r `G=−4πρ₀r`.	refined S(Q)/G(r) + bg
`pair_distribution_g` / `total_correlation_T` / `radial_distribution_R`	Convention family from G(r) + ρ_0.	g/T/R

> **Packaging note.** `midas_pdf` is currently **vendored** in `midas_gui/_vendor/` and loaded through `midas_gui/pdf_backend.py`, which also installs a small `midas_hkls.absorption` compatibility shim for `midas-hkls < 0.5.0`. Once `midas-pdf` is published and `midas-hkls>=0.5.0` is available the vendored copy and the shim are removed.

---

10. Tab 7 — Texture / Pole Figure *(work in progress)*

Per-ring azimuthal analysis for preferred orientation. Controls: calibration source, sample frame, R/η bins, ring index, χ (tilt) / ϕ (rotation). **Compute Pole Figure** integrates to a cake and extracts I(η) at the selected ring; the right panel shows the stereographic pole figure and the raw I(η). **Save pole figure (.pol)** exports POPLA format.

---

11. Pump Probe (time-resolved / TR-XRD)

Analyses time-resolved (pump-probe) diffraction the way the TRR group does: a folder of raw detector frames is pooled by a filename prefix, the pump-probe **delay** is parsed from each name, every frame is integrated to I(q) with the **MIDAS engine** (the same core as Batch Integrate, driven by a calibration), repeats at each delay are averaged, and a reference (mean of the pre-time-zero / negative delays) is subtracted to give **ΔI(q, delay)**.

File-naming / pooling. Frames follow `PREFIX–<fshw>fshw<delay>delay<id>.tif`, where `PREFIX` (e.g. `Ex01_Sa01_Sc17` = Experiment / Sample / Scan) is one opaque grouping key. **Scan folder** globs `PREFIX*.tif` in the loaded folder, parses `fshw` and `delay` (seconds) from each name, and reports how many frames and unique delays were found. The delay sign is flipped on load so positive delay = after the pump.

The tab uses the same three-panel layout as Batch Integrate: **data loader** (left), **settings** (middle), **plots** (right).

Left — data loader (the shared loader panel, as on Calibrate / Batch): - **Data** — the folder of raw frames. Defaults to the bundled TRR test set in `test_data_pump_probe/detimages/` (125 Pilatus2M frames; a large, git-ignored local asset), so the tab is populated on open. An index range / stride can subset the pooled frames. - **Dark / Bright / Background** — per-frame field corrections (compute each field, then it is applied to every frame before integration). - **Mask** — a mask file and/or the mask from the Mask Builder tab. For the TRR data the tab pre-loads `invert_mask.tif` (0 = valid pixel, 1 = bad pixel / module gap), which matches MIDAS's convention that non-zero pixels are masked out.

Middle — settings. - **Calibration source** — *From Tab 2* (the live calibration) or *From file* (`calibration.json` / `paramstest.txt` / `.poni`). Same convention as Batch Integrate. For the shipped TRR test data the field defaults to a ready MIDAS `paramstest` (`Ex01_Sa01_Sc17_midas.txt`, converted from the dataset's pyFAI/Fit2D geometry), so the tab integrates out of the box. - **Data pooling (TRR)** — the pooling **prefix** + **Scan folder** (the folder itself comes from the loader on the left). - **Integration** — kernel, R/η bins, the plot axis (Q / 2θ / R), and optional Q-uniform binning. Identical engine and options to Batch Integrate. - **Physics corrections** — polarization / solid-angle. - **Pump-probe options** — the reference-delay set (defaults to all negative delays; multi-select to override), optional per-pattern normalization over a q-window, the ΔI colour range (auto or fixed ±) and the diverging colormap.

Views (right panel). Every axis shows **real physical values**, not indices — the radial axis is in Q ($Å^{-1}$), 2θ (°) or R (px) per the plot-axis selector, and delays are in seconds. 1. **ΔI heatmap** — ΔI over the radial axis (a true, continuous Q/2θ/R axis) versus delay, diverging colour centred at zero, with a labelled ΔI colour-bar and the reference I(q) lineout beside it (shared radial axis). Delays span several decades and are irregular, so they are laid out as columns labelled with their real delay values. 2. **ΔI vs q** — one curve per delay, coloured along a rainbow by delay. With ≤ 8 delays each curve is named in the legend; with more, a continuous **delay colour-bar** (min → max, in seconds) replaces the legend. 3. **Kinetics** — ΔI versus delay for user-defined q-bands (**Add band** over a q range); x-axis as real delay (**linear**), **log** (post-t_0 delays, natural for the decade-wide delay range), or **rank**. 4. **Mean patterns** — the averaged I(q) at each delay plus the dashed reference, with an optional **±1σ band** (spread of I(q) across delays) and a **Log Y** toggle to reveal weak features across the full dynamic range; a signal-level / stability check.

All views use a publication-style white theme with large, clearly-labelled axes, readable legends and a subtle grid. A toolbar above the plots sets the **draw** mode (lines / lines+points / points) and the **line**, **point (sym)** and **font** sizes (the –/+ groups), as on the Batch Integrate tab. Pan and zoom are bounded to the data so you cannot lose the plot area.

Integration runs off the GUI thread (`PumpProbeWorker`) with a progress bar and **Abort**. There is no peak fitting — peak position / width / area are read from the plots, matching the reference workflow.

---

12. Results & Export *(work in progress)*

Session summary + one-click export. Checkboxes select which products (calibration.json, paramstest.txt, mask.tif, integrated profiles, G(r), pole figures, session log) to copy to an output directory. A provenance block (package versions, geometry hash, mask fraction, correction flags) can be copied to the clipboard for a Methods section.

---

13. Common UI Conventions

- **Browse = load.** Selecting a file/folder (or pressing Enter in a path field) loads it immediately; there are no separate "Load" buttons. HDF5 dataset / frame-index changes reload automatically.
- **No accidental scroll changes.** Spin boxes and drop-downs ignore the mouse wheel — values change only by clicking/typing; the wheel scrolls the panel instead.
- **Readable right-click menus.** pyqtgraph plot context menus use the dark theme.
- **Dark theme, orange accent** (Dioptas-inspired); off-white text, light input fields.
- **Sample-to-detector distance is entered/shown in mm** (Data Viewer, Calibrate seed, Preferences ▸ Geometry). Internally — all calculations — and in written calibration files (`paramstest.txt`, `calibration.json`) it is always in **microns**; the mm↔µm conversion happens only at the display boundary. The config key remains `lsd_um` (µm).

---

14. Packaging, Deployment & Diagnostics

Packaging. `midas-gui` is a MIDAS-style package: `pyproject.toml` (BSD-3-Clause), a `tests/` smoke suite, and `release.sh` for cutting versioned releases (see `RELEASING.md`). Once published it can join the `midas-suite` meta-package as an optional `gui` extra (`pip install midas-suite[gui]`).

Deployment on another system. Clone the repo, create the conda environment (`environment.yml`), and run `midas-gui`. The MIDAS analysis backends are installed via the `pip:` section of `environment.yml` (editable from a local MIDAS checkout, from a git subdirectory, or from a private index). `midas_gui/_paths.py` is an inert stub in an installed package (no `sys.path` manipulation).

Crash diagnostics. On startup the app installs a logging exception hook and `faulthandler`; any uncaught Python exception or native fault is written to `~/midas_gui_error.log` and shown in a dialog. Installing the hook also prevents PyQt5 from hard-aborting on an exception inside a slot. Each tab is built in isolation — a tab that fails becomes an error placeholder instead of taking the window down. If the app ever "pops up and dies" (typically on Windows), send that log file.

15. Configuration & Defaults

Every default in the GUI — detector geometry, data/output paths, ring-simulation **materials**, **calibrants**, the **pixel-preset** and **K-edge** menus, the calibration / integration **algorithms**, and **which tabs are visible** — can be set **without editing code**, via a single per-user JSON config that overrides the shipped built-in defaults.

Where it lives

One per-user file, auto-located per OS: `~/.config/midas_gui/config.json` (Linux), `~/Library/Application Support/midas_gui/config.json` (macOS), `%APPDATA%\midas_gui\config.json` (Windows). If it's absent, the shipped built-in defaults are used. Sharing between machines/users is done by exporting/importing a JSON file (below) — copy it wherever you like.

Profiles — Settings ▸ Preferences… ▸ Profile row

The top of the Preferences dialog has a **Profile** row — `Profile: [combo ▼] [New…] [Duplicate…] [Rename…] [Delete]` — for keeping several independent sets of defaults (e.g. one per beamline) and switching between them: - Each profile is its own config file under `<config dir>/profiles/<name>.json`; the active one is remembered in `<config dir>/profile_meta.json`. Existing single-config installs are migrated transparently into a profile named **Default** the first time this runs — no data is lost. - **New…** seeds a blank profile from the shipped built-in defaults. - **Duplicate…** seeds a new profile from whatever is currently shown in the dialog (including unsaved edits), then switches to it. - **Rename… / Delete** — Delete refuses to remove the last remaining profile; deleting the active profile switches to another one first. - Switching profiles (via the combo) prompts to confirm if the dialog has unsaved edits, then reloads the dialog's fields and the live `DEFAULT_*` globals from the new profile — most values apply immediately; a few (e.g. viewer step sizes set at widget-construction time) need a restart, and you're offered one. - The "Local config: …" label under the row becomes **"Profile '<name>': <path>"** so you always know which file you're editing.

Editing — Settings ▸ Preferences…

The **Settings** menu opens **Preferences…**, a dialog whose tabs are **pre-filled with the full shipped defaults** so you edit from a complete starting point: - **Geometry** — λ, pixel size, Lsd, beam centre. - **Data Viewer** — the up/down-arrow step size for each field in the Data Viewer's Ring simulation card (λ, max 2θ, Lsd, pixel size, BC_y/BC_z, ty/tz). Shipped defaults: 0.01 Å, 1°, 1 mm, 0.1 µm, 1 px, 0.1° respectively. - **Paths** — default data / calibration / output files & folders. - **Materials** / **Calibrants** — add / remove / modify (name + lattice + SG). - **Devices** — the detector devices offered in the Data Viewer's **Live Data** PV dropdown (name, prefix, PVA suffix). The live PV is built as `prefix + PVA suffix`. Ships pre-filled with the 20-ID-D detectors (`oryx20idd`, `s20idPil`, `pg4`, `gh1s`, `s20varex1`), all with PVA suffix `Pva1:Image`; add / remove / edit rows for your own beamline's devices. - **Menus** — the pixel-size presets and K-edge foils. - **Algorithms** — default calibration pipeline, integration kernel, output format, error model, colormap/theme. - **Tabs** — which tabs are visible. Data Viewer, Mask Builder, Calibrate and Batch Integrate are always shown (their boxes are locked on); tick/untick the rest. **Tab visibility applies immediately** on save (no restart), unlike the other settings. - **Display** — the **interface scale**, a whole-application zoom (layout *and* fonts) for HiDPI / 4K monitors where the default looks too small. Pick a value or a preset (100 % ≈ 1080p, 150 % ≈ 1440p, 200 % ≈ 4K); it applies after a restart (you're offered one on save). Also reachable from **Settings ▸ Interface scaling…**. It is implemented with Qt's `QT_SCALE_FACTOR`, so the whole layout scales uniformly.

Buttons: **Save as my defaults** (write your config), **Save current GUI state** (capture the Data Viewer's live λ/pixel/Lsd/BC into the fields), **Save config to JSON… / Load config (JSON)…** (export/import a config to share), and **Reset to shipped defaults** (delete your config). Also on the menu: **Open config folder** and **Reload config.** Saving writes your per-user file; **changes take effect on the next launch**.

File format

```
{
  "geometry": { "wavelength_A": 0.39, "pixel_um": 75.0, "lsd_um": 121000.0,
                "bc_y": 10.0, "bc_z": 10.0,
                "pixel_presets": [["Eiger", 75.0], ["Pilatus", 172.0]],
                "k_edge_foils": [["Au", 80.725], ["Pb", 88.005]] },
  "viewer_steps": { "wavelength": 0.01, "two_theta": 1.0, "lsd_mm": 1.0,
                    "pixel": 0.1, "bc": 1.0, "tilt": 0.1 },
  "materials":  { "Ni (FCC)":
        {"a":3.5238,"b":3.5238,"c":3.5238,"alpha":90,"beta":90,"gamma":90,"sg":225} },
  "calibrants": { "CeO2":
        {"a":5.4116,"b":5.4116,"c":5.4116,"alpha":90,"beta":90,"gamma":90,"sg":225} },
  "devices": [ {"name": "s20varex1", "prefix": "20IDFF:", "pva_suffix": "Pva1:Image"} ],
  "paths": { "nickel_h5": "/data/mygroup/sample.h5", "calib_file":
        "/data/mygroup/calibration.json" },
  "ui": { "calibration_pipeline": "one_shot", "integration_kernel": "subpixel2",
          "output_format": "csv", "azimuthal_method": "poisson", "plot_theme": "hot",
          "visible_tabs": ["Calib. Refinement", "Pump Probe"],
          "ui_scale": 1.0 }
}
```

- `ui.visible_tabs` lists the **optional** tabs to show (the four always-on tabs are implicit). The shipped default is `["Calib. Refinement", "Pump Probe"]` — Corrections, PDF Analysis, Texture and Results & Export are hidden until you add them. Edit it from **Preferences ▸ Tabs**.
- `ui.ui_scale` is the whole-interface zoom (0.5–4.0) applied at startup via `QT_SCALE_FACTOR`; ~1.5 for 1440p, ~2.0 for 4K. Edit it from **Preferences ▸ Display** or **Settings ▸ Interface scaling…** (takes effect on restart).
- When a **list section is present it fully replaces** that built-in list — so `materials`, `calibrants`, `devices`, `pixel_presets` and `k_edge_foils` in the file are your complete lists. (The Preferences dialog pre-fills them with the shipped entries, so you always start from the full set; use **Reset to shipped defaults** to get them back.) `sg` is the space-group number.
- Geometry scalars, `paths` and `ui` are simple overrides; any section or key may be omitted. A malformed config is ignored (built-ins are used) rather than blocking startup.
- A minimal template lives at `documentation/config.example.json`; the step-by-step guide is `documentation/config_gui.md`.

---

16. File ▸ Save/Load GUI State

Beyond a saved **profile** of defaults (§15), the **File** menu can capture the **live, in-progress state of every tab** — every field you've typed or picked, across all 10 tabs — to one JSON file, so a session can be closed and resumed later exactly where it left off.

- **File ▸ Save GUI State…** (`Ctrl+S`) — the first time in a session, prompts for a destination (default `midas_session.json`); every subsequent `Ctrl+S` **silently overwrites that same file** (no dialog) as long as the session hasn't loaded/saved a different file since. **File ▸ Save GUI State As…** (`Ctrl+Shift+S`) always prompts for a destination, and makes *that* file the target for future plain `Ctrl+S`. Either way, a completion dialog lists which tabs (if any) failed to save.
- **File ▸ Load GUI State…** (`Ctrl+O`) — asks for confirmation (loading overwrites every tab's current values), then pick a previously saved file. A completion dialog reports any tabs present in the file that no longer exist, or that failed to restore. The tab that was active at save time is re-selected.

What is restored automatically

Every field (text boxes, spin boxes, checkboxes, combo/dropdown selections) in every tab is restored as typed. In addition, **path-backed data is reloaded from disk**, the same way it loads when you type/browse to a path by hand — images, masks-by-path, dark/bright/background frames, and HDF5 datasets all re-read their file automatically after a state load (guarded so a moved/deleted file is skipped quietly rather than popping a warning).

What is *not* re-run

Loading a state **does not re-run any long-running pipeline** — Calibrate's Fit, Batch Integrate, the PDF transform, and Calibration Refinement all keep their inputs restored but require **one manual click of the tab's own Run/Fit button** to reproduce their result. This keeps a load fast and avoids silently kicking off a multi-minute job in the background.

Sidecar files for in-progress derived data

Two tabs can hold computed data that hasn't been exported to a file of its own yet — a drawn/computed **mask** (Mask Builder) and a just-fit **calibration result** (Calibrate). Saving a GUI state writes small sidecar files next to it so this in-progress work isn't silently lost: - `<state file stem>_mask.tif` — the current in-memory mask, if any; reloaded automatically on the next Load. - `<state file stem>_calibration.json` — the last fit result's flattened fields, kept for the record and to reseed the manual/seed geometry fields. This does **not** reconstruct the in-memory fitted result object (it isn't a plain, re-loadable structure) — re-run **Fit** after loading to reproduce it.

For bugs or questions, see the MIDAS GUI repository.